COMP7710

Programming Fundamentals

Today's lecture

Generic Programming

Course Convenor
Bernardo Pereira Nunes

Australian National University

All lectures are recorded and made available on Canvas

Agenda

> Generic Programming
> Type Parameters
> Inheritance Rules
> Wildcard Types
> Exercises

What is Generic Programming?

> Writing code that can be reused for objects of many different types
>> instead of separate classes for each type, one generic class handles them all

> All use the same class with different type parameters:

```
% ArrayList<String>
% ArrayList<File>
% ArrayList<Employee>
```


Before Generics: Generic Programming was achieved with inheritance

> `ArrayList` used `Object` references; but, two serious problems:

```
public class ArrayList { // before generic classes
    private Object[] elementData;
    public Object get(int i) { ... }
    public void add(Object o) { ... }
}
```

① Unsafe Casts Required

```
% ArrayList files = new ArrayList();
% String filename = (String) files.get(i);
```

You must cast on every retrieval; verbose and error-prone;

May fail during runtime with `ClassCastException` if the object is not actually a `String`.

② No Type Checking

```
% files.add(Path.of(". . ."));
% files.add(new File("...")); // compiles, all classes inherits from Object
% files.add(123);
```

Generics: Type Parameters as a solution

> A type parameter specifies the type of object a generic class or method can store or use
>> `ArrayList<String>` uses `String` as the *type parameter*

```
% // Using var (type inferred)
var files = new ArrayList<String>();

% // Using diamond syntax: type inferred from variable
ArrayList<String> files = new ArrayList<>();
```

intent is clear: stores a list of Strings

it can only add `Strings`

no cast needed

```
// Compiler knows it's a String
String filename = files.get(0);
```

compile-time safety

```
// Compiler error caught early
files.add(new File("..."));
```

Three levels of Genetic Programming

1) Use Generic Classes
> Use `ArrayList<String>`, `HashMap<K,V>` etc. without worrying about the internals
> Most application programmers stay here

2) Understand & Troubleshoot
> Learn enough to decode confusing error messages when mixing different generic classes
> Read compiler diagnostics confidently

3) Write Generic Code
> Implement your own generic classes and methods
> Anticipate all future uses
> Design with wildcards and bounded type parameters

```
% ArrayList<Employee> employees; // employees.addAll(managers);
% ArrayList<Manager> managers;   // managers.addAll(employees);
```


Defining a simple Generic Class

> A generic class can have multiple type variables
> It allows fields to have different types

```
public class Pair<T, U> {
    private T first;
    private U second;
...
}
```

> A **generic class** is a class with one or more type variables
>> type variables allow the class to work with different data types

Example

> a `Pair<T>` class can store two objects of the same type:

```
public class Pair<T> {
    private T first;
    private T second;
    // constructors
    public Pair() { first = null; second = null; }
    public Pair(T first, T second) { this.first = first; this.second = second; }
    // getters
    public T getFirst() { return first; }
    public T getSecond() { return second; }
    // setters
    public void setFirst(T newValue) { first = newValue; }
    public void setSecond(T newValue) { second = newValue; }
}
```

focus on generics without being distracted by data storage details

return types, types of fields and local variable

Defining a simple Generic Class

Use uppercase letters for type variables.

The Java library uses:
E for the element type of a collection,
K and V for key and value types of a table, and
T (and the neighboring letters U and S, if necessary) for any type at all

jshell

```
% Pair<String> p = new Pair<>();
% p.setFirst("COMP7710");
% p.setSecond("Bernardo");
...
```

Defining a Generic Method

Ordinary class (no type)

Generic Method
<T> means the method works with any type

jshell
```
% public class ArrayUtils {
    public static <T> T getMiddle(T... a) {
        return a[a.length / 2];
    }

    public static void main(String[] args) {
        String middle = ArrayUtils.getMiddle("John", "Mary", "Peter");
        Integer number = ArrayUtils.getMiddle(10, 20, 30, 40, 50);

        IO.println(middle); // Mary
        IO.println(number); // 30
    }
}
```

Java infers the type automatically, but it can sometimes slow compilation. We can specify the type explicitly:

```
ArrayUtils.<String>getMiddle(
                    "John", "Mary", "Peter");
```

S

Bounds for Type Variables

A generic type can have multiple interface bounds, but only one class bound, which must appear first in the bounds list.

> What does this code do?
> Is there anything wrong with this code?

```
% <T> T min(T[] a) {
    if (a == null || a.length == 0) return null;
    T smallest = a[0];
    for (int i = 1; i < a.length; i++) {
        if (smallest.compareTo(a[i]) > 0) smallest = a[i];
    }
    return smallest;
}
```

Variable smallest has type T

How do we know that the class to which T belongs has a `compareTo` method?

a class or a method needs to place restrictions on type variables

```
<T extends Comparable> T min(T[] a) . . .
```

Only classes that implement `Comparable` are allowed; otherwise, a compile time error occurs.
Multiple bounds are allowed:
`<T extends Comparable & Serializable>`

Generic Exceptions

What you CANNOT do:
- ❌ **Catch** a type variable: `catch (T e)` will not compile
- ❌ **Throw or catch** objects of a generic class
- ❌ **Extend** Throwable with a generic class: `class Problem<T> extends Exception` is illegal

> A method can declare a throws clause using a type variable, enabling flexible exception propagation through generic functional interfaces

```
interface Task<T extends Throwable> {
    void run() throws T;
}
```

> propagating the Generic Exception Methods using the interface can propagate the exception with the same type:

```
<T extends Throwable> void repeat(Task<T> t, int n) throws T {
    for (int i = 0; i < n; i++) t.run();
}
```

> since `Thread.sleep` throws the checked `InterruptedException`, calling `repeat(() -> Thread.sleep(100), 10)` throws the same exception

Type Erasure

Type erasure removes generic type parameters during compilation
> A generic type automatically gets a **corresponding raw type**
> Type variables are replaced by:
>> their first bound, or
>> `Object` if unbounded

Generics and the JVM

> The JVM does not store separate generic types at runtime
> All generic objects become ordinary classes after compilation
> The compiler removes type parameter information through type erasure
> This allows older JVMs to run code compiled with generics

```
public class Pair { // Pair<T> -> Pair; T is erased to Object since it is unbounded
    private Object first;
    private Object second;
    // constructors
    public Pair() { first = null; second = null; }
    public Pair(Object first, Object second) { this.first = first; this.second = second; }
    // getters
    public Object getFirst() { return first; }
    public Object getSecond() { return second; }
    // setters
    public void setFirst(Object newValue) { first = newValue; }
    public void setSecond(Object newValue) { second = newValue; }
}
```

The result is an ordinary class, similar to how classes were implemented before generics in Java.

Type Erasure

replaces type variables with the first bound

```
public class Interval<T extends Comparable & Serializable> implements Serializable {
    private T lower;
    private T upper;
    . . .
    public Interval(T first, T second) {
        if (first.compareTo(second) <= 0) { lower = first; upper = second; }
        else { lower = second; upper = first; }
    }
}

// after Type Erasure

public class Interval implements Serializable {
    private Comparable lower;
    private Comparable upper;
    . . .
    public Interval(Comparable first, Comparable second) { . . . }
}
```

if the bounds are switched (`T extends Serializable & Comparable`), the raw type replaces `T` with `Serializable`, and the compiler inserts casts to `Comparable` when needed.

Type Erasure: Translating Generic Expressions

After type erasure, the **compiler automatically inserts casts** when retrieving generic values or accessing generic fields

```
// method call
Pair<Employee> buddies = ...;
Employee buddy = buddies.getFirst();

// after type erasure (conceptually)
Employee buddy = (Employee) buddies.getFirst();
```

the compiler does not literally rewrite the Java source like this, but it behaves as if those casts were inserted automatically

```
// field access
Employee buddy = buddies.first;

// after type erasure (conceptually)
Employee buddy = (Employee) buddies.first;
```

Type Erasure: Translating Generic Methods

Generic methods are erased to their bound type

```
// method
<T extends Comparable> T min(T[] a)

// after type erasure (conceptually)
Comparable min(Comparable[] a)
```

Problem: Erasure and Polymorphism

```
// before type erasure
class DateInterval extends Pair<LocalDate> {
    public void setSecond(LocalDate second) {
        ...
    }
}
```

```
// after type erasure
class DateInterval extends Pair {
    public void setSecond(LocalDate second) {
        ...
    }
}
```

These are NOT the same signature

But `Pair` still has: `public void setSecond(Object second)`

Type Erasure: Translating Generic Methods

Problem: Erasure and Polymorphism

```
// before type erasure
class DateInterval extends Pair<LocalDate> {
    public void setSecond(LocalDate second) {
        ...
    }
}
```

```
// after type erasure
class DateInterval extends Pair {
    public void setSecond(LocalDate second) {
        ...
    }
}
```

These are NOT the same signature

But `Pair` still has: `public void setSecond(Object second)`

The compiler creates a **bridge method** to **preserve polymorphism**, otherwise overriding would break and polymorphism would fail

```
public void setSecond(Object second) {
    setSecond((LocalDate) second);
}
// ensures it correctly calls: DateInterval.setSecond(LocalDate)
Pair<LocalDate> pair = interval;
pair.setSecond(aDate);
```

Bridge method

preserves polymorphism after type erasure
performs the necessary cast automatically
redirects the call to the real method

Inheritance Rules for Generic Types

> Even if `Manager extends Employee`, this does NOT mean: `Pair<Manager> extends Pair<Employee>`

```
//illegal
Pair<Employee> buddies = new Pair<Manager>(ceo, cfo);
```

> Why?

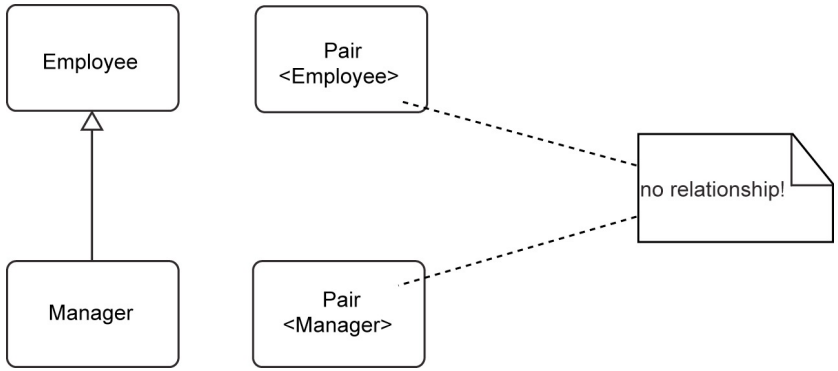
```
var managerBuddies = new Pair<Manager>(ceo, cfo);

// illegal, but suppose it were allowed:
Pair<Employee> employeeBuddies = managerBuddies;

employeeBuddies.setFirst(lowlyEmployee);
```

Consequence...

> `employeeBuddies` and `managerBuddies` refer to the same object
> `setFirst` would allow inserting an `Employee` into a `Pair<Manager>`
> this would corrupt the original `Pair<Manager>` and break type safety

Important: Generics vs Arrays!
Generic types are invariant.
Arrays are covariant in Java:
(`Manager[] → Employee[]`), but the JVM checks assignments at runtime (*Generics use Type Erasure, JVM does not know the type*) and throws `ArrayStoreException` for invalid inserts

`Pair<S>` and `Pair<T>` are unrelated types, even if `S` and `T` are related



No inheritance relationship between `Pair` classes
(source: Core Java Vol 1 Book)

Wildcard Types

> A wildcard type in Java is a generic type that uses ? to represent an unknown type parameter,
>> allowing flexible and type-safe operations on related generic types

> Suppose we want to print two employees:

```
% void printBuddies(Pair<Employee> p) {
    Employee first  = p.getFirst();
    Employee second = p.getSecond();

    IO.println(first.getName() + " and " + second.getName() + " are buddies.");
}
```

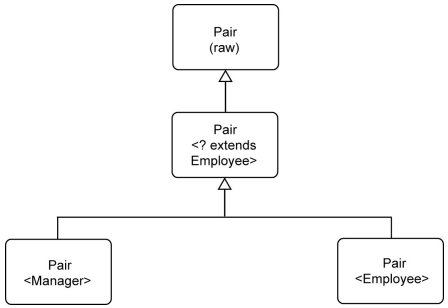
> This method **cannot accept:** `Pair<Manager>` (as we have seen before `Pair<Manager>` ≠ `Pair<Employee>`)

Wildcard Types

> The solution is simple:

```
% void printBuddies(Pair<? extends Employee> p)
```

> The type `Pair<Manager>` is a subtype of `Pair<? extends Employee>`



Subtype relationships with wildcards
(source: Core Java Vol 1 Book)

```
var managerBuddies = new Pair<Manager>(ceo, cfo);
Pair<? extends Employee> wildcardBuddies = managerBuddies; // OK
wildcardBuddies.setFirst(lowlyEmployee); // compile-time error
```

`setFirst` cannot be safely called because the compiler does not know the exact subtype inside `Pair<? extends Employee>` (it could be `Manager`, `Executive`, etc.), so it rejects all non-null arguments.

`getFirst` is safe because the returned value is guaranteed to be some subtype of `Employee`, so it can always be assigned to an `Employee` reference. This is the key idea of bounded wildcards: reading is safe, writing is not.

Supertype Bounds for Wildcards

> A wildcard can also specify a supertype bound:

```
% ? super Executive (writing)
```

> Meaning: any type that is a superclass of `Executive (Object, Employee, Manager, Executive)`

> Opposite behaviour of `? Extends (reading)`

Example

```
% Pair<? super Executive>

// Conceptually behaves like:
void setFirst(? super Executive)
? super Executive getFirst()
```

Writing is safe because every valid type can store an `Executive` (e.g., `Pair<Object>, Pair<Employee>, Pair<Manager>, Pair<Executive>` can hold an `Executive`

Reading is not safe because compiler only knows the result is some supertype of `Executive`. It might actually be `Object, Employee, Manager`, so only assignment to `Object` is guaranteed safe

Unbounded Wildcards

> A wildcard can also specify a supertype bound:

```
% Pair<?>
```

> Meaning: a `Pair` of some unknown type

> Raw type `Pair` allows `pair.setFirst(anyObject); // unsafe`

Example

```
% Pair<?>

// Conceptually behaves like:
? getFirst()
void setFirst(?)
```

```
Object obj = pair.getFirst(); // safely read as Object
```

The compiler only knows the value is some unknown type, so it can only guarantee `Object`.

```
pair.setFirst(obj); // ERROR, cannot safely write values
```

Not allowed, the actual type inside the pair is unknown.
Only this is allowed: `pair.setFirst(null);`

Why Use: ?
Useful when the actual type does not matter

Example

```
boolean hasNulls(Pair<?> p) {
    return p.getFirst() == null
        || p.getSecond() == null;
}
```


Wildcard Capture

For more restrictions and limitations:
See section 8.5 in the Core Java, Vol I book

> Suppose we want to swap elements in a pair:

```
% void swap(Pair<?> p)
```

> We cannot use ? as a real type:

```
% ? t = p.getFirst();
```

Error, a wildcard is not a type variable

```
% p.setFirst(p.getSecond());
```

It fails, the compiler knows the pair contains some unknown type, but it does not know which one

Solution: use a generic helper

So inside `swapHelper`: `Pair<T>` has a consistent type, making the swap safe

```
% <T> void swapHelper(Pair<T> p) {
    T t = p.getFirst();
    p.setFirst(p.getSecond());
    p.setSecond(t);
  }
```

```
void swap(Pair<?> p) {
    swapHelper(p);
}
```

the helper method "captures" the wildcard
the compiler treats ? as a specific but unknown type T

<Exercises>

Why were generics introduced in Java?

A. To make programs run faster

B. To remove inheritance from Java

C. To provide compile-time type safety and avoid unsafe casts

D. To replace arrays completely

<Exercises>

What problem could occur before generics when retrieving objects from an ArrayList?

A. StackOverflowError
B. ClassCastException
C. IOException
D. ArrayStoreException

<Exercises>

What does the type parameter in ArrayList<String> specify?

A. The maximum size of the list
B. The memory location of the list
C. The superclass of ArrayList
D. The type of objects stored in the list

<Exercises>

Which statement correctly creates a generic ArrayList<String>?

A. ArrayList files = new ArrayList();

B. ArrayList<String> files = new ArrayList<>();

C. ArrayList<> files = new ArrayList<String>();

D. ArrayList<int> files = new ArrayList<>();

<Exercises>

What is the benefit of this generic method?

A. It only works with Strings
B. It removes arrays from Java
C. It guarantees runtime polymorphism
D. It works with many different types

<Exercises>

Which bounded type declaration correctly restricts T to comparable objects?

A. <T extends Comparable>
B. <T implements Comparable>
C. <T super Comparable>
D. <T comparable>

<Exercises>

With Pair<? extends Employee>, which operation is safe?

A. Writing Employee objects
B. Writing Manager objects
C. Reading values as Employee
D. Replacing the Pair type

<Exercises>

What does Pair<?> represent?

A. A Pair of Strings only
B. A raw type
C. A Pair of some unknown type
D. A Pair without methods

<Exercises>

Why does this line produce an error?

```
? t = p.getFirst();
```

A. Wildcards are not real type variables
B. getFirst returns void
C. ? means Object
D. Java forbids local variables

MEME

Would you like to go for a <T>?

Not my Cup<T>

Thank you

Australian National University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)
CRICOS PROVIDER CODE: 00120C